LAB 07 – Advanced REST API Integration with Swift

Revised Self-Guided Edition

© 2026 Dara Aghamirkarimi — Educational Material
Designed, reviewed, and validated by the instructor with the assistance of generative artificial intelligence tools. The content has been adapted and enhanced to align with the course's learning objectives.

---

1. Introduction

In this laboratory, you will build a complete client-server application composed of three main elements:

1. A REST API developed with Python and Flask.
2. A SQLite database containing student information.
3. A Swift console application that communicates with the REST API.

The Swift application does not connect directly to the SQLite database. Instead, it sends HTTP requests to the Flask server.

The Flask server receives each request, accesses the database, and returns a JSON response.

The complete communication process is:

```
User
  |
  | Selects a menu option
  v
Swift console application
  |
  | HTTP request
  v
Flask REST API
  |
  | SQL query
  v
SQLite database
  |
  | Database result
  v
Flask REST API
  |
  | HTTP response containing JSON
  v
Swift console application
  |
  | JSON decoding
```

```
  v
Information displayed to the user
```

This architecture is commonly called a client-server architecture.

---

2. Learning Objectives

By the end of this laboratory, you should be able to:

- Explain client-server architecture.
- Explain the request-response cycle.
- Create a REST API with Flask.
- Store persistent data in SQLite.
- Use the HTTP methods GET, POST, PUT, and DELETE.
- Send and receive JSON.
- Create Swift models that conform to `Codable`.
- Encode Swift objects using `JSONEncoder`.
- Decode JSON using `JSONDecoder`.
- Send HTTP requests using `URLSession`.
- Use asynchronous programming with `async` and `await`.
- Use `throws`, `try`, `do`, and `catch`.
- Interpret HTTP status codes.
- Create custom Swift errors.
- Validate information on the client and server.
- Organize a Swift program into multiple files.
- Separate networking logic from application logic.
- Filter and sort Swift arrays.
- Calculate statistics from collections.
- Compile several Swift files into one executable application.
- Create a VS Code task using `tasks.json`.

---

3. The Roles of the Different Technologies

Technology	Role
Swift console application	REST client and user interface
Flask	HTTP server and REST API
SQLite	Persistent database

Technology	Role
JSON	Data format exchanged between the client and server
Thunder Client	Independent REST API testing tool
VS Code	Code editor and development environment
`tasks.json`	Automates compilation and execution of the Swift application

Important distinction

The Swift menu and Thunder Client are separate clients.

Thunder Client is used to test REST endpoints manually.

The Swift console application is the client that students develop.

Both clients communicate with the same Flask server and the same SQLite database.

For example, if you add a student using Thunder Client, that student will also appear in the Swift application.

---

4. Application Architecture

The application uses the following architecture:

```
+----------------------------+
| Swift Console Application  |
|                            |
| main.swift                 |
| Student.swift              |
| APIService.swift           |
| APIError.swift             |
+-------------+--------------+
              |
              | HTTP requests and JSON
              v
+----------------------------+
| Flask REST API             |
|                            |
| app.py                     |
+-------------+--------------+
              |
              | SQL queries
```

```
              v
+---------------------------+
| SQLite Database           |
|                           |
| students.db               |
+---------------------------+
```

The responsibilities are separated:

- `main.swift` manages the menu and user interaction.
- `Student.swift` defines the data models.
- `APIService.swift` manages HTTP communication.
- `APIError.swift` defines the possible networking errors.
- `app.py` receives HTTP requests and communicates with SQLite.
- `students.db` stores the student records permanently.

This separation makes the application easier to understand, test, and maintain.

---

5. REST Endpoints

The Flask API contains the following endpoints:

Method	Endpoint	Purpose
GET	`/students`	Retrieve all students
GET	`/students/{id}`	Retrieve one student
POST	`/students`	Add a student
PUT	`/students/{id}`	Update a student
DELETE	`/students/{id}`	Delete a student
GET	`/students/program/{program}`	Retrieve students from a particular program
GET	`/students/statistics/average-age`	Calculate the average student age

Examples:

```
GET http://127.0.0.1:5000/students
```

```
GET http://127.0.0.1:5000/students/2
```

```
DELETE http://127.0.0.1:5000/students/3
```

PART A – PROJECT PREPARATION

6. Project Structure

Create a folder named:

```
LAB07_ADVANCED_REST_SWIFT
```

Inside the folder, create the following structure:

```
LAB07_ADVANCED_REST_SWIFT
│
├── .vscode
│   └── tasks.json
│
├── api
│   ├── app.py
│   └── students.db
│
└── swift
    ├── main.swift
    ├── Student.swift
    ├── APIService.swift
    └── APIError.swift
```

What does “project root” mean?

The project root is the main folder that contains all the other project folders.

In this laboratory, the root is:

```
LAB07_ADVANCED_REST_SWIFT
```

The `.vscode` folder must be directly inside the root.

It must not be placed inside `swift`.

Correct:

```
LAB07_ADVANCED_REST_SWIFT/.vscode/tasks.json
```

Incorrect:

```
LAB07_ADVANCED_REST_SWIFT/swift/.vscode/tasks.json
```

When opening the project in VS Code, open the complete folder:

```
LAB07_ADVANCED_REST_SWIFT
```

Do not open only the `swift` folder because the build task contains:

```
cd swift
```

If VS Code is already inside the `swift` folder, the task would attempt to find another `swift` folder inside it.

---

7. Environment Requirements

You need:

- Windows 10 or Windows 11
- Visual Studio Code
- Swift for Windows
- Visual Studio Community
- The C++ workload required by Swift
- Windows SDK
- Python 3
- Flask
- SQLite
- Swift extension for VS Code
- Thunder Client extension for VS Code

PART B – THE FLASK SERVER

8. Install Flask

Open the project in VS Code.

Open a terminal and move to the `api` folder:

```
Bash

cd api
```

Install Flask:

```bash
pip install flask
```

Verify the installation:

```bash
pip show flask
```

SQLite is included with Python. You do not need to install a separate SQLite Python package.

---

9. Complete Flask Server

Create the following file:

```
api/app.py
```

The provided file may originally be named `app(1).py`. Rename it to:

```
app.py
```

Use the following complete code.

```python
from dbm import error

from flask import Flask, request, jsonify
import sqlite3

app = Flask(__name__)


def init_database():

    connection = sqlite3.connect("students.db")

    cursor = connection.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS students
        (
```

```
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            program TEXT NOT NULL,
            age INTEGER NOT NULL
        )
    """)

    cursor.execute("SELECT COUNT(*) FROM students")

    count = cursor.fetchone()[0]

    if count == 0:

        students = [
            ("Alice", "Computer Science", 21),
            ("Bob", "Business", 23),
            ("Charlie", "Computer Science", 20)
        ]

        cursor.executemany(
            """
            INSERT INTO students(name, program, age)
            VALUES (?, ?, ?)
            """,
            students
        )

    connection.commit()
    connection.close()


def get_connection():

    connection = sqlite3.connect("students.db")

    connection.row_factory = sqlite3.Row

    return connection


@app.route("/students", methods=["GET"])
def get_students():

    connection = get_connection()

    students = connection.execute(
        "SELECT * FROM students"
    ).fetchall()

    connection.close()

    result = []
```

```
    for student in students:

        result.append({
            "id": student["id"],
            "name": student["name"],
            "program": student["program"],
            "age": student["age"]
        })

    return jsonify(result)


@app.route("/students/<int:id>", methods=["GET"])
def get_student(id):

    connection = get_connection()

    student = connection.execute(
        "SELECT * FROM students WHERE id = ?",
        (id,)
    ).fetchone()

    connection.close()

    if student is None:

        return jsonify({
            "message": "Student not found"
        }), 404

    return jsonify({
        "id": student["id"],
        "name": student["name"],
        "program": student["program"],
        "age": student["age"]
    })


def validate_student(data):

    if "name" not in data:
        return "Name is required"

    if "program" not in data:
        return "Program is required"

    if "age" not in data:
        return "Age is required"

    if data["name"].strip() == "":
        return "Name cannot be empty"
```

```
    if data["program"].strip() == "":
        return "Program cannot be empty"

    if not isinstance(data["age"], int):
        return "Age must be an integer"

    if data["age"] < 16 or data["age"] > 100:
        return "Age must be between 16 and 100"

    return None


@app.route("/students", methods=["POST"])
def add_student():

    data = request.get_json()

    error = validate_student(data)

    if error is not None:

        return jsonify({
            "message": error
        }), 400

    connection = get_connection()

    cursor = connection.cursor()

    cursor.execute(
        """
        INSERT INTO students(name, program, age)
        VALUES (?, ?, ?)
        """,
        (
            data["name"],
            data["program"],
            data["age"]
        )
    )

    connection.commit()

    new_id = cursor.lastrowid

    connection.close()

    return jsonify({
        "id": new_id,
        "name": data["name"],
        "program": data["program"],
```

```
        "age": data["age"]
    }), 201


@app.route("/students/<int:id>", methods=["PUT"])
def update_student(id):

    data = request.get_json()

    error = validate_student(data)

    if error is not None:

        return jsonify({
            "message": error
        }), 400

    connection = get_connection()

    student = connection.execute(
        "SELECT * FROM students WHERE id = ?",
        (id,)
    ).fetchone()

    if student is None:

        connection.close()

        return jsonify({
            "message": "Student not found"
        }), 404

    connection.execute(
        """
        UPDATE students
        SET name = ?, program = ?, age = ?
        WHERE id = ?
        """,
        (
            data["name"],
            data["program"],
            data["age"],
            id
        )
    )

    connection.commit()
    connection.close()

    return jsonify({
        "id": id,
        "name": data["name"],
```

```
        "program": data["program"],
        "age": data["age"]
    })


@app.route("/students/<int:id>", methods=["DELETE"])
def delete_student(id):

    connection = get_connection()

    student = connection.execute(
        "SELECT * FROM students WHERE id = ?",
        (id,)
    ).fetchone()

    if student is None:

        connection.close()

        return jsonify({
            "message": "Student not found"
        }), 404

    connection.execute(
        "DELETE FROM students WHERE id = ?",
        (id,)
    )

    connection.commit()
    connection.close()

    return jsonify({
        "message": "Student deleted successfully"
    })


@app.route("/students/program/<program>", methods=["GET"])
def get_students_by_program(program):

    connection = get_connection()

    students = connection.execute(
        """
        SELECT * FROM students
        WHERE LOWER(program) = LOWER(?)
        """,
        (program,)
    ).fetchall()

    connection.close()

    result = []
```

```
    for student in students:

        result.append({
            "id": student["id"],
            "name": student["name"],
            "program": student["program"],
            "age": student["age"]
        })

    return jsonify(result)


@app.route("/students/statistics/average-age", methods=["GET"])
def get_average_age():

    connection = get_connection()

    result = connection.execute(
        """
        SELECT AVG(age) AS averageAge
        FROM students
        """
    ).fetchone()

    connection.close()

    return jsonify({
        "averageAge": result["averageAge"]
    })


if __name__ == "__main__":

    init database()
```

---

10. Understanding the Flask Code

10.1 Imports

Python Run

```
from flask import Flask, request, jsonify
import sqlite3
```

`Flask` creates the server application.

`request` gives the program access to information sent by the client.

For example:

```python
data = request.get_json()
```

This reads the JSON body sent by Swift or Thunder Client.

`jsonify` converts Python dictionaries and lists into JSON responses.

`sqlite3` provides access to SQLite databases.

The following import appears in the supplied final code:

```python
from dbm import error
```

It is not used anywhere in the application. Removing it would not change the application's behaviour.

10.2 Creating the Flask Application

```python
app = Flask(__name__)
```

This creates the Flask application.

The `app` object is used to define the routes and start the server.

10.3 Initializing the Database

```python
def init_database():
```

This function prepares the database when the server starts.

```python
connection = sqlite3.connect("students.db")
```

This opens the `students.db` database.

If the file does not exist, SQLite creates it.

Because `"students.db"` is a relative path, the file is created in the current working directory.

This is why you should start Flask from the `api` folder:

Bash

```
cd api
python app.py
```

Otherwise, Python may create another `students.db` file in a different directory.

10.4 Creating the Table

Python Run

```
cursor.execute("""
    CREATE TABLE IF NOT EXISTS students
    (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        program TEXT NOT NULL,
        age INTEGER NOT NULL
    )
""")
```

`CREATE TABLE IF NOT EXISTS` creates the table only if it does not already exist.

This means the command can run every time the server starts without deleting the existing table.

The columns are:

Column	Type	Description
`id`	INTEGER	Unique student identifier
`name`	TEXT	Student’s name
`program`	TEXT	Student’s program

Column	Type	Description
`age`	INTEGER	Student's age

`PRIMARY KEY` means that `id` uniquely identifies each row.

`AUTOINCREMENT` means that SQLite generates the new ID automatically.

`NOT NULL` means that the database does not accept a missing value for that column.

10.5 Inserting Initial Data

Python Run

```python
cursor.execute("SELECT COUNT(*) FROM students")
count = cursor.fetchone()[0]
```

`COUNT(*)` calculates the number of rows in the table.

If the number is zero, the application inserts three initial students:

Python Run

```python
if count == 0:
```

The starter values are stored in a list of tuples:

Python Run

```python
students = [
    ("Alice", "Computer Science", 21),
    ("Bob", "Business", 23),
    ("Charlie", "Computer Science", 20)
]
```

`executemany` executes the same SQL command for every tuple:

Python Run

```python
cursor.executemany(
    """
    INSERT INTO students(name, program, age)
    VALUES (?, ?, ?)
    """,
    students
    )
```

The question marks are SQL parameters.

The values are transmitted separately from the SQL command. This is safer than manually combining user values with SQL text.

Parameterized queries help prevent SQL injection.

10.6 Commit and Close

Python Run

```
connection.commit()
connection.close()
```

`commit()` permanently saves database modifications.

Without `commit()`, an insertion, update, or deletion might not be permanently saved.

`close()` closes the database connection.

Every opened connection should eventually be closed.

10.7 Database Connection Helper

Python Run

```
def get_connection():

    connection = sqlite3.connect("students.db")

    connection.row_factory = sqlite3.Row

    return connection
```

Instead of repeating the same connection code inside every route, the program creates a helper function.

The line:

Python Run

```
connection.row_factory = sqlite3.Row
```

allows columns to be accessed by name.

Without `sqlite3.Row`, a row might be accessed like this:

```python
student[1]
```

With `sqlite3.Row`, it can be accessed more clearly:

```python
student["name"]
```

Named column access is easier to read and maintain.

---

11. GET All Students

The route is:

```python
@app.route("/students", methods=["GET"])
```

It connects this URL:

```
GET /students
```

to this function:

```python
def get_students():
```

The query retrieves every student:

```python
students = connection.execute(
    "SELECT * FROM students"
).fetchall()
```

`fetchall()` returns all matching rows.

The rows are converted into dictionaries:

Python Run

```python
result.append({
    "id": student["id"],
    "name": student["name"],
    "program": student["program"],
    "age": student["age"]
})
```

Finally:

Python Run

```python
return jsonify(result)
```

converts the list to JSON.

An example response is:

JSON

```json
[
  {
    "id": 1,
    "name": "Alice",
    "program": "Computer Science",
    "age": 21
  },
  {
    "id": 2,
    "name": "Bob",
    "program": "Business",
    "age": 23
  }
]
```

---

12. GET One Student

The route contains a dynamic ID:

Python Run

```python
@app.route("/students/<int:id>", methods=["GET"])
```

For this request:

```
GET /students/2
```

Flask converts `2` into an integer and passes it to:

```python
def get_student(id):
```

The SQL query is:

```python
student = connection.execute(
    "SELECT * FROM students WHERE id = ?",
    (id,)
).fetchone()
```

The expression:

```python
(id,)
```

is a one-element Python tuple.

The comma is necessary.

If no student is found:

```python
if student is None:
```

the API returns:

```json
{
  "message": "Student not found"
}
```

with status code `404`.

---

13. Server-Side Validation

The following function validates incoming student data:

```python
def validate_student(data):
```

It checks:

- Whether `name` exists.
- Whether `program` exists.
- Whether `age` exists.
- Whether the name is empty.
- Whether the program is empty.
- Whether age is an integer.
- Whether age is between 16 and 100.

When the data is valid, the function returns:

```python
None
```

When the data is invalid, it returns an error message.

For example:

```python
return "Age must be between 16 and 100"
```

The server must validate information even if the Swift application already validates it.

The Flask API may receive requests from:

- The Swift application
- Thunder Client
- A web application
- A mobile application
- Another external client

The server cannot assume that every client performs validation correctly.

---

14. POST – Add a Student

The route is:

Python Run

```python
@app.route("/students", methods=["POST"])
```

The server reads the JSON body:

Python Run

```python
data = request.get_json()
```

It validates the data:

Python Run

```python
error = validate_student(data)
```

If an error exists:

Python Run

```python
if error is not None:
```

the server returns status `400 Bad Request`.

The insertion uses parameters:

Python Run

```python
cursor.execute(
    """
    INSERT INTO students(name, program, age)
    VALUES (?, ?, ?)
    """,
    (
        data["name"],
        data["program"],
        data["age"]
    )
)
```

After the insertion:

Python Run

```python
new_id = cursor.lastrowid
```

retrieves the ID generated by SQLite.

The response contains the complete student:

JSON

```
{
  "id": 4,
  "name": "Diana",
  "program": "Computer Science",
  "age": 22
}
```

The status is:

```
201 Created
```

---

15. PUT – Update a Student

The route is:

Python Run

```
@app.route("/students/<int:id>", methods=["PUT"])
```

PUT replaces the complete student information.

The client must send:

JSON

```
{
  "name": "Updated Name",
  "program": "Updated Program",
  "age": 25
}
```

The server first validates the body.

It then verifies that the student exists.

If the student does not exist, it returns `404`.

The update query is:

Python Run

```
connection.execute(
    """
    UPDATE students
    SET name = ?, program = ?, age = ?
    WHERE id = ?
    """,
    (
        data["name"],
        data["program"],
        data["age"],
        id
    )
)
```

The response contains the updated student.

---

16. DELETE – Delete a Student

The route is:

Python Run

```
@app.route("/students/<int:id>", methods=["DELETE"])
```

The server verifies that the student exists.

It then runs:

Python Run

```
connection.execute(
    "DELETE FROM students WHERE id = ?",
    (id,)
)
```

The response is:

JSON

```
{
  "message": "Student deleted successfully"
}
```

Notice that DELETE does not return a `Student`.

It returns an object containing a message.

This is why the Swift project needs the `MessageResponse` structure.

---

17. Filter Students by Program

The route is:

```python
@app.route("/students/program/<program>", methods=["GET"])
```

Example:

```
GET /students/program/Computer%20Science
```

The SQL query is:

```sql
SELECT * FROM students
WHERE LOWER(program) = LOWER(?)
```

`LOWER` converts both values to lowercase.

Therefore, these values are treated as equivalent:

```
Computer Science
computer science
COMPUTER SCIENCE
```

The response is an array because several students can belong to the same program.

---

18. Average Age Endpoint

The route is:

```python
@app.route("/students/statistics/average-age", methods=["GET"])
```

The SQL query is:

```sql
```

```
SELECT AVG(age) AS averageAge
FROM students
```

`AVG(age)` calculates the average.

`AS averageAge` gives the result a JSON-compatible name that matches the Swift model:

Swift

```
struct AverageAgeResponse: Codable {
    let averageAge: Double
}
```

The JSON response is similar to:

JSON

```
{
  "averageAge": 21.33
}
```

---

19. Start the Flask Server

Inside the `api` folder, run:

Bash

```
python app.py
```

You should see an address similar to:

```
http://127.0.0.1:5000
```

Keep this terminal open.

The Flask server must continue running while you use Thunder Client or the Swift application.

PART C – TESTING WITH THUNDER CLIENT

20. Why Use Thunder Client?

Thunder Client lets you test Flask independently from Swift.

If an endpoint fails in Thunder Client, the problem is probably in Flask or SQLite.

If the endpoint works in Thunder Client but fails in Swift, the problem is probably in the Swift client.

This separation makes debugging easier.

---

21. Test GET All Students

Use:

```
GET http://127.0.0.1:5000/students
```

Expected status:

```
200 OK
```

Expected response:

JSON

```
[
  {
    "id": 1,
    "name": "Alice",
    "program": "Computer Science",
    "age": 21
  }
]
```

---

22. Test POST

Use:

```
POST http://127.0.0.1:5000/students
```

Select a JSON body:

JSON

```
{
  "name": "Diana",
  "program": "Computer Science",
```

```
  "age": 22
}
```

Make sure the request uses:

```
Content-Type: application/json
```

Expected status:

```
201 Created
```

---

23. Test PUT

Use:

```
PUT http://127.0.0.1:5000/students/1
```

Body:

JSON

```
{
  "name": "Alice Johnson",
  "program": "Software Development",
  "age": 22
}
```

---

24. Test DELETE

Use:

```
DELETE http://127.0.0.1:5000/students/1
```

Expected response:

JSON

```
{
  "message": "Student deleted successfully"
}
```

PART D – SWIFT DATA MODELS

25. Student.swift

Create:

```
swift/Student.swift
```

Add:

Swift

```swift
import Foundation

struct Student: Codable {

    let id: Int
    let name: String
    let program: String
    let age: Int
}


struct NewStudent: Codable {

    let name: String
    let program: String
    let age: Int
}


struct MessageResponse: Codable {

    let message: String
}


struct AverageAgeResponse: Codable {

    let averageAge: Double
}
```

---

26. Understanding the Swift Models

26.1 Student

Swift

```swift
struct Student: Codable {
    let id: Int
    let name: String
    let program: String
    let age: Int
}
```

`Student` represents a complete student returned by the server.

It includes `id` because SQLite generates an ID for every stored student.

26.2 NewStudent

Swift

```swift
struct NewStudent: Codable {
    let name: String
    let program: String
    let age: Int
}
```

`NewStudent` represents information sent when creating or updating a student.

It does not contain `id`.

When creating a student, the client should not invent an ID. SQLite generates it.

26.3 MessageResponse

Swift

```swift
struct MessageResponse: Codable {
    let message: String
}
```

This model corresponds to JSON such as:

JSON

```json
{
  "message": "Student deleted successfully"
}
```

26.4 AverageAgeResponse

Swift

```swift
struct AverageAgeResponse: Codable {
    let averageAge: Double
}
```

This corresponds to:

JSON

```json
{
  "averageAge": 21.33
}
```

26.5 What Is Codable?

`Codable` combines two protocols:

```
Codable = Encodable + Decodable
```

`Encodable` allows Swift values to be converted to JSON.

Swift

```swift
JSONEncoder().encode(newStudent)
```

`Decodable` allows JSON to be converted into Swift values.

Swift

```swift
JSONDecoder().decode(Student.self, from: data)
```

The Swift properties must match the JSON keys.

JSON:

JSON

```json
{
  "id": 1,
  "name": "Alice",
  "program": "Computer Science",
```

```
  "age": 21
}
```

Swift:

```swift
struct Student: Codable {
    let id: Int
    let name: String
    let program: String
    let age: Int
}
```

PART E – CUSTOM ERRORS

27. APIError.swift

Create:

```
swift/APIError.swift
```

Add:

```swift
enum APIError: Error, CustomStringConvertible {

    case invalidURL
    case invalidResponse
    case badRequest
    case notFound
    case serverError
    case decodingError
    case encodingError
    case networkError

    var description: String {

        switch self {

        case .invalidURL:
            return "Invalid URL"

        case .invalidResponse:
            return "Invalid server response"

        case .badRequest:
```

```
            return "Invalid data sent to server"

        case .notFound:
            return "Resource not found"

        case .serverError:
            return "Server error"

        case .decodingError:
            return "Failed to decode data"

        case .encodingError:
            return "Failed to encode data"

        case .networkError:
            return "Network communication failed"
        }
    }
}
```

---

28. Understanding APIError

The enum conforms to:

Swift

```
Error
```

This allows its cases to be thrown:

Swift

```
throw APIError.notFound
```

It also conforms to:

Swift

```
CustomStringConvertible
```

This protocol requires:

Swift

```
var description: String
```

When the application runs:

Swift

```swift
print("Error: \(error)")
```

the user sees a readable message.

For example:

```
Error: Resource not found
```

instead of a low-level technical error.

Error categories

Error	Meaning
`invalidURL`	Swift could not create the URL
`invalidResponse`	Response was not valid HTTP or had an unsupported status
`badRequest`	Server returned HTTP 400
`notFound`	Server returned HTTP 404
`serverError`	Server returned a status from 500 to 599
`decodingError`	JSON could not be converted to a Swift model
`encodingError`	Swift data could not be converted to JSON
`networkError`	Communication with the server failed

PART F – NETWORKING SERVICE

29. APIService.swift

Create:

```
swift/APIService.swift
```

Add:

Swift

```swift
import Foundation
import FoundationNetworking

class APIService {

    private let baseURL = "http://127.0.0.1:5000"


    private func decode<T: Decodable>(
        type: T.Type,
        from data: Data
    ) throws -> T {

        do {

            return try JSONDecoder().decode(
                type,
                from: data
            )

        } catch {

            throw error
        }
    }


    private func decode<T: Decodable>(
        _ type: T.Type,
        from data: Data
    ) throws -> T {

        do {

            return try JSONDecoder().decode(
                type,
                from: data
            )

        } catch {

            throw APIError.decodingError
        }
    }


    private func validateResponse(
        _ response: URLResponse
    ) throws {
```

```
        guard let httpResponse =
                response as? HTTPURLResponse else {

            throw APIError.invalidResponse
        }

        switch httpResponse.statusCode {

        case 200...299:
            return

        case 400:
            throw APIError.badRequest

        case 404:
            throw APIError.notFound

        case 500...599:
            throw APIError.serverError

        default:
            throw APIError.invalidResponse
        }
    }


    func getStudents() async throws -> [Student] {

        guard let url = URL(
            string: "\(baseURL)/students"
        ) else {

            throw APIError.invalidURL
        }

        do {

            let (data, response) =
                try await URLSession.shared.data(
                    from: url
                )

            try validateResponse(response)

            return try decode(
                [Student].self,
                from: data
            )

        } catch let error as APIError {

            throw error
```

```
        } catch {

            throw APIError.networkError
        }
    }


    func getStudent(id: Int) async throws -> Student {

        guard let url = URL(
            string: "\(baseURL)/students/\(id)"
        ) else {

            throw APIError.invalidURL
        }

        do {

            let (data, response) =
                try await URLSession.shared.data(
                    from: url
                )

            try validateResponse(response)

            return try decode(
                Student.self,
                from: data
            )

        } catch let error as APIError {

            throw error

        } catch {

            throw APIError.networkError
        }
    }


    func addStudent(
        _ newStudent: NewStudent
    ) async throws -> Student {

        guard let url = URL(
            string: "\(baseURL)/students"
        ) else {

            throw APIError.invalidURL
        }
```

```
    var request = URLRequest(url: url)

    request.httpMethod = "POST"

    request.setValue(
        "application/json",
        forHTTPHeaderField: "Content-Type"
    )

    do {

        request.httpBody =
            try JSONEncoder().encode(
                newStudent
            )

    } catch {

        throw APIError.encodingError
    }

    do {

        let (data, response) =
            try await URLSession.shared.data(
                for: request
            )

        try validateResponse(response)

        return try decode(
            Student.self,
            from: data
        )

    } catch let error as APIError {

        throw error

    } catch {

        throw APIError.networkError
    }
}


func updateStudent(
    id: Int,
    student: NewStudent
) async throws -> Student {
```

```
        guard let url = URL(
            string: "\(baseURL)/students/\(id)"
        ) else {

            throw APIError.invalidURL
        }

        var request = URLRequest(url: url)

        request.httpMethod = "PUT"

        request.setValue(
            "application/json",
            forHTTPHeaderField: "Content-Type"
        )

        do {

            request.httpBody =
                try JSONEncoder().encode(
                    student
                )

        } catch {

            throw APIError.encodingError
        }

        do {

            let (data, response) =
                try await URLSession.shared.data(
                    for: request
                )

            try validateResponse(response)

            return try decode(
                Student.self,
                from: data
            )

        } catch let error as APIError {

            throw error

        } catch {

            throw APIError.networkError
        }
    }
```

```
func deleteStudent(
    id: Int
) async throws -> String {

    guard let url = URL(
        string: "\(baseURL)/students/\(id)"
    ) else {

        throw APIError.invalidURL
    }

    var request = URLRequest(url: url)

    request.httpMethod = "DELETE"

    do {

        let (data, response) =
            try await URLSession.shared.data(
                for: request
            )

        try validateResponse(response)

        let result = try decode(
            MessageResponse.self,
            from: data
        )

        return result.message

    } catch let error as APIError {

        throw error

    } catch {

        throw APIError.networkError
    }
}


func validateStudent(
    name: String,
    program: String,
    age: Int
) -> Bool {

    if name
        .trimmingCharacters(in: .whitespaces)
        .isEmpty {
```

```
            print("Name cannot be empty")

            return false
        }

        if program
            .trimmingCharacters(in: .whitespaces)
            .isEmpty {

            print("Program cannot be empty")

            return false
        }

        if age < 16 || age > 100 {

            print("Age must be between 16 and 100")

            return false
        }

        return true
    }


    func getStudentsByProgram(
        program: String
    ) async throws -> [Student] {

        guard let encodedProgram =
                program.addingPercentEncoding(
                    withAllowedCharacters: .urlPathAllowed
                ) else {

            throw APIError.invalidURL
        }

        guard let url = URL(
            string:
                "\(baseURL)/students/program/\(encodedProgram)"
        ) else {

            throw APIError.invalidURL
        }

        do {

            let (data, response) =
                try await URLSession.shared.data(
                    from: url
                )
```

```
            try validateResponse(response)

            return try decode(
                [Student].self,
                from: data
            )

        } catch let error as APIError {

            throw error

        } catch {

            throw APIError.networkError
        }
```

---

30. Understanding APIService

30.1 Why Create APIService?

Without `APIService`, `main.swift` would need to repeat:

- URL construction
- HTTP request preparation
- JSON encoding
- JSON decoding
- Status-code validation
- Error conversion

`APIService` centralizes all these responsibilities.

The menu can simply call:

Swift

```
try await api.getStudents()
```

instead of manually creating every request.

This follows the principle of separation of responsibilities.

30.2 Foundation and FoundationNetworking

Swift

```
import Foundation
import FoundationNetworking
```

`Foundation` provides:

- `URL`
- `Data`
- `JSONEncoder`
- `JSONDecoder`
- String formatting
- General utility types

`FoundationNetworking` provides networking classes on systems where networking is separated from Foundation, particularly Linux.

Including both imports improves portability.

30.3 Base URL

Swift

```
private let baseURL = "http://127.0.0.1:5000"
```

This is the root address of the Flask server.

`127.0.0.1` means the current computer.

It is also called:

```
localhost
```

The port is:

```
5000
```

because Flask uses port 5000 by default.

`private` means that the property can only be accessed from inside `APIService`.

30.4 Generic Decode Function

```swift
private func decode<T: Decodable>(
    _ type: T.Type,
    from data: Data
) throws -> T
```

`T` is a generic type.

It represents any type conforming to `Decodable`.

For example:

```swift
Student.self
```

means that `T` is `Student`.

```swift
[Student].self
```

means that `T` is `[Student]`.

The expression:

```swift
T.Type
```

means that the function receives the type itself rather than an existing object.

The function can therefore decode many different response types.

30.5 Two Decode Functions

The final code contains two decode functions:

```swift
decode(type:from:)
```

and:

Swift

```
decode(_:from:)
```

The first version requires an external `type:` label:

Swift

```
decode(
    type: Student.self,
    from: data
)
```

The second removes the first external label using `_`:

Swift

```
decode(
    Student.self,
    from: data
)
```

All current API methods use the second version.

Therefore, the first version is currently redundant.

There is another difference:

The first version rethrows the original error:

Swift

```
throw error
```

The second converts every decoding failure to:

Swift

```
throw APIError.decodingError
```

The second version produces a consistent custom error for the rest of the application.

30.6 Validate the HTTP Response

Swift

```
private func validateResponse(
    _ response: URLResponse
) throws
```

`URLSession` initially returns a general `URLResponse`.

To access the HTTP status code, the program converts it to:

```
Swift

HTTPURLResponse
```

The cast is performed with:

```
Swift

guard let httpResponse =
        response as? HTTPURLResponse else {
    throw APIError.invalidResponse
}
```

If the response is not an HTTP response, the function throws an error.

30.7 HTTP Status Codes

```
Swift

switch httpResponse.statusCode
```

Success:

```
Swift

case 200...299:
    return
```

The complete range from 200 to 299 represents successful HTTP responses.

Examples:

```
200 OK
201 Created
204 No Content
```

Bad request:

```swift
case 400:
    throw APIError.badRequest
```

Missing resource:

```swift
case 404:
    throw APIError.notFound
```

Server error:

```swift
case 500...599:
    throw APIError.serverError
```

Every other unexpected response becomes:

```swift
APIError.invalidResponse
```

Important concept

A status such as `404` is not necessarily a network failure.

The network communication may have succeeded perfectly:

1. Swift sent the request.
2. Flask received it.
3. Flask processed it.
4. Flask returned a response.
5. The response said the resource did not exist.

Therefore, URLSession can successfully return a response containing status `404`.

The application must inspect the status code separately.

30.8 GET Request

A simple GET can use:

```swift
URLSession.shared.data(from: url)
```

It returns a tuple:

```swift
(data, response)
```

`data` contains the response body.

`response` contains metadata such as the HTTP status.

The method then:

1. Validates the response.
2. Decodes the JSON.
3. Returns the Swift model.

30.9 POST and PUT Requests

POST and PUT require a configured `URLRequest`.

```swift
var request = URLRequest(url: url)
```

Set the method:

```swift
request.httpMethod = "POST"
```

or:

```swift
request.httpMethod = "PUT"
```

Set the content type:

```swift

```

```
request.setValue(
    "application/json",
    forHTTPHeaderField: "Content-Type"
)
```

This tells Flask that the body contains JSON.

Encode the Swift object:

```Swift
request.httpBody =
    try JSONEncoder().encode(newStudent)
```

The result is binary `Data` containing JSON.

30.10 DELETE Request

DELETE needs a custom request because the HTTP method must be set:

```Swift
request.httpMethod = "DELETE"
```

The response is decoded as:

```Swift
MessageResponse.self
```

This is because Flask returns:

```JSON
{
  "message": "Student deleted successfully"
}
```

The API method returns only the String:

```Swift
return result.message
```

30.11 async and await

Network requests require time.

The result is not immediately available.

The method is declared:

```swift
async
```

The caller uses:

```swift
await
```

Example:

```swift
let students = try await api.getStudents()
```

`await` means:

> Wait for this asynchronous operation to complete before using its result.

It does not mean the program freezes the entire operating system thread. Swift's concurrency system manages the waiting operation.

30.12 throws and try

A method declared with:

```swift
throws
```

can stop and produce an error.

The caller must use:

```swift
try
```

Example:

```swift
try await api.getStudents()
```

The call is placed inside `do-catch`:

```swift
do {

    let students =
        try await api.getStudents()

} catch {

    print("Error: \(error)")
}
```

30.13 Why Two catch Blocks?

The methods use:

```swift
catch let error as APIError {
    throw error
} catch {
    throw APIError.networkError
}
```

The first catch preserves errors that were intentionally created by the application.

For example:

```swift
APIError.notFound
APIError.decodingError
APIError.badRequest
```

The second catch converts every other unexpected error into:

```swift
```

```
APIError.networkError
```

Without the first catch, a `404` could incorrectly become a generic network error.

30.14 URL Encoding

A program name may contain a space:

```
Computer Science
```

Spaces cannot always be inserted directly into a URL path.

The code uses:

```
Swift

program.addingPercentEncoding(
    withAllowedCharacters: .urlPathAllowed
)
```

A space is transformed into a safe URL representation.

For example:

```
Computer Science
```

can become:

```
Computer%20Science
```

The final endpoint becomes:

```
/students/program/Computer%20Science
```

30.15 Currently Unused Code

`APIService` contains:

```
Swift

func validateStudent(...)
```

However, `main.swift` also contains a global function named:

Swift

```
func validateStudent(...)
```

The menu uses the global function in `main.swift`.

Therefore, the `APIService` validation function is currently unused.

`APIService` also contains:

Swift

```
getStudentsByProgram(program:)
```

However, menu option 6 currently calls:

Swift

```
getStudents()
```

and filters the complete array locally.

This is valid, but students should recognize that a server-side filtering method is already available.

PART G – CONSOLE APPLICATION

31. main.swift

Create:

```
swift/main.swift
```

Add:

Swift

```
let api = APIService()


func validateStudent(
    name: String,
    program: String,
    age: Int
) -> Bool {
```

```
    if name
        .trimmingCharacters(in: .whitespaces)
        .isEmpty {

        print("Name cannot be empty.")

        return false
    }

    if program
        .trimmingCharacters(in: .whitespaces)
        .isEmpty {

        print("Program cannot be empty.")

        return false
    }

    if age < 16 || age > 100 {

        print("Age must be between 16 and 100.")

        return false
    }

    return true
}


func displayStatistics(
    students: [Student]
) {

    if students.isEmpty {

        print("No students available.")

        return
    }

    let ages = students.map {
        $0.age
    }

    let youngest = ages.min() ?? 0

    let oldest = ages.max() ?? 0

    let totalAge = ages.reduce(0, +)

    let average =
        Double(totalAge) /
```

```
        Double(ages.count)

    print(
        "Number of students: \(students.count)"
    )

    print(
        "Youngest student age: \(youngest)"
    )

    print(
        "Oldest student age: \(oldest)"
    )

    print(
        "Average age: \(String(format: "%.2f", average))"
    )
}


var running = true


while running {

    print("""
    ==================================
           STUDENT MANAGEMENT
    ==================================
    1. Display all students
    2. Search student by ID
    3. Add student
    4. Update student
    5. Delete student
    6. Filter by program
    7. Sort by age
    8. Display statistics
    0. Exit

    Choose an option:
    """)

    let choice = readLine()

    switch choice {

    case "1":

        do {

            let students =
                try await api.getStudents()
```

```
            if students.isEmpty {

                print("No students found.")

            } else {

                for student in students {

                    print(
                        "\(student.id) - " +
                        "\(student.name) - " +
                        "\(student.program) - " +
                        "\(student.age)"
                    )
                }
            }

        } catch {

            print("Error: \(error)")
        }


    case "2":

        print("Enter student ID:")

        guard let input = readLine(),
              let id = Int(input) else {

            print("Invalid ID.")

            continue
        }

        do {

            let student =
                try await api.getStudent(id: id)

            print("ID: \(student.id)")
            print("Name: \(student.name)")
            print("Program: \(student.program)")
            print("Age: \(student.age)")

        } catch {

            print("Error: \(error)")
        }
```

```
    case "3":

        print("Enter name:")

        let name = readLine() ?? ""

        print("Enter program:")

        let program = readLine() ?? ""

        print("Enter age:")

        guard let ageInput = readLine(),
              let age = Int(ageInput) else {

            print("Age must be an integer.")

            continue
        }

        if validateStudent(
            name: name,
            program: program,
            age: age
        ) {

            let newStudent = NewStudent(
                name: name,
                program: program,
                age: age
            )

            do {

                let student =
                    try await api.addStudent(
                        newStudent
                    )

                print(
                    "Student created successfully."
                )

                print(
                    "\(student.id) - " +
                    "\(student.name) - " +
                    "\(student.program) - " +
                    "\(student.age)"
                )

            } catch {
```

```
                print("Error: \(error)")
            }
        }


    case "4":

        print("Enter student ID:")

        guard let idInput = readLine(),
              let id = Int(idInput) else {

            print("Invalid ID.")

            continue
        }

        print("Enter new name:")

        let name = readLine() ?? ""

        print("Enter new program:")

        let program = readLine() ?? ""

        print("Enter new age:")

        guard let ageInput = readLine(),
              let age = Int(ageInput) else {

            print("Age must be an integer.")

            continue
        }

        if validateStudent(
            name: name,
            program: program,
            age: age
        ) {

            let updatedData = NewStudent(
                name: name,
                program: program,
                age: age
            )

            do {

                let student =
                    try await api.updateStudent(
                        id: id,
```

```
                    student: updatedData
                )

            print(
                "Student updated successfully."
            )

            print(
                "\(student.id) - " +
                "\(student.name) - " +
                "\(student.program) - " +
                "\(student.age)"
            )

        } catch {

            print("Error: \(error)")
        }
    }


case "5":

    print("Enter student ID:")

    guard let input = readLine(),
          let id = Int(input) else {

        print("Invalid ID.")

        continue
    }

    do {

        let message =
            try await api.deleteStudent(
                id: id
            )

        print(message)

    } catch {

        print("Error: \(error)")
    }


case "6":

    print("Enter program:")
```

```
        let program = readLine() ?? ""

        do {

            let students =
                try await api.getStudents()

            let result = students.filter {

                $0.program.lowercased() ==
                program.lowercased()
            }

            if result.isEmpty {

                print(
                    "No students found for this program."
                )

            } else {

                for student in result {

                    print(
                        "\(student.id) - " +
                        "\(student.name) - " +
                        "\(student.program) - " +
                        "\(student.age)"
                    )
                }
            }

        } catch {

            print("Error: \(error)")
        }


    case "7":

        do {

            let students =
                try await api.getStudents()

            let sortedStudents =
                students.sorted {

                    $0.age < $1.age
                }

            for student in sortedStudents {
```

```
                print(
                    "\(student.id) - " +
                    "\(student.name) - " +
                    "\(student.program) - " +
                    "\(student.age)"
                )
            }

        } catch {

            print("Error: \(error)")
        }


    case "8":

        do {

            let students =
                try await api.getStudents()

            displayStatistics(
                students: students
            )

        } catch {

            print("Error: \(error)")
        }


    case "0":

        running = false

        print("Goodbye!")


    default:
```

---

32. Understanding main.swift

32.1 APIService Instance

Swift

```
let api = APIService()
```

This creates one `APIService` object.

The menu uses the object for every server request.

32.2 Client-Side Validation

The function verifies:

- Name is not empty.
- Program is not empty.
- Age is between 16 and 100.

```Swift
name.trimmingCharacters(in: .whitespaces)
```

removes spaces before and after the text.

For example:

```
"      "
```

contains characters, but after removing whitespace, it is empty.

The function returns:

```Swift
true
```

when the information is valid.

It returns:

```Swift
false
```

when the information is invalid.

32.3 Why Validate Twice?

The project validates data:

1. In Swift before sending it.
2. In Flask after receiving it.

Client-side validation improves the user experience because errors can be shown before making a request.

Server-side validation protects the database because the API cannot trust every possible client.

32.4 Statistics Function

Swift

```
func displayStatistics(
    students: [Student]
)
```

The function receives an array of students.

Before performing calculations, it verifies that the array is not empty:

Swift

```
if students.isEmpty {
    print("No students available.")
    return
}
```

This prevents division by zero.

32.5 map

Swift

```
let ages = students.map {
    $0.age
}
```

`map` transforms every element.

The original collection is:

Swift

```
[Student]
```

The new collection is:

```
Swift

[Int]
```

Example:

```
Students: Alice 21, Bob 23, Charlie 20
```

After `map`:

```
Swift

[21, 23, 20]
```

`$0` represents the current student.

32.6 min and max

```
Swift

let youngest = ages.min() ?? 0
let oldest = ages.max() ?? 0
```

`min()` finds the smallest age.

`max()` finds the largest age.

These functions return optionals because the array might be empty.

The operator:

```
Swift

??
```

is the nil-coalescing operator.

It provides a default value if the result is `nil`.

32.7 reduce

Swift

```
let totalAge = ages.reduce(0, +)
```

`reduce` combines all array values into one result.

It starts at zero:

```
0
```

Then adds every age.

For:

Swift

```
[21, 23, 20]
```

the calculation is:

```
0 + 21 + 23 + 20 = 64
```

32.8 Calculating the Average

Swift

```
let average =
    Double(totalAge) /
    Double(ages.count)
```

Both values are converted to `Double`.

Without this conversion, integer division could remove the decimal portion.

For example:

```
64 / 3
```

with integers could produce:

```
21
```

With `Double`, it produces approximately:

```
21.3333
```

32.9 Formatting to Two Decimal Places

Swift

```swift
String(format: "%.2f", average)
```

`%.2f` means:

- `f`: floating-point value
- `.2`: two digits after the decimal point

Example:

```
21.333333
```

becomes:

```
21.33
```

32.10 Main Loop

Swift

```swift
var running = true
```

The menu continues while `running` is `true`.

Swift

```swift
while running {
```

When the user selects zero:

Swift

```swift
running = false
```

The loop stops.

32.11 readLine

```swift
let choice = readLine()
```

`readLine()` waits for keyboard input.

Its return type is:

```swift
String?
```

It is optional because keyboard input may not be available.

32.12 switch

```swift
switch choice
```

Each `case` corresponds to a menu option.

```swift
case "1":
```

handles option 1.

```swift
default:
```

handles every value that does not match an existing case.

32.13 guard and continue

Example:

```swift
```

```
guard let input = readLine(),
      let id = Int(input) else {

    print("Invalid ID.")

    continue
}
```

The first condition unwraps the optional returned by `readLine()`.

The second attempts to convert the string to `Int`.

If either condition fails, the `else` block executes.

```
    Swift

continue
```

immediately begins the next iteration of the menu loop.

---

33. Menu Option 1 – Display All Students

```
    Swift

let students =
    try await api.getStudents()
```

This sends:

```
GET /students
```

The response is decoded as:

```
    Swift

[Student]
```

The loop displays each student:

```
    Swift

for student in students
```

---

34. Menu Option 2 – Search by ID

The application reads an ID and calls:

```swift
try await api.getStudent(id: id)
```

This sends:

```
GET /students/{id}
```

If the student exists, Swift receives a `Student`.

If the student does not exist, Flask returns `404`, and `APIService` throws:

```swift
APIError.notFound
```

---

35. Menu Option 3 – Add a Student

The application:

1. Reads the name.
2. Reads the program.
3. Reads the age.
4. Validates the information.
5. Creates a `NewStudent`.
6. Sends a POST request.
7. Receives the complete created `Student`.

```swift
let newStudent = NewStudent(
    name: name,
    program: program,
    age: age
)
```

The server generates the ID.

---

36. Menu Option 4 – Update a Student

The application first reads the ID of the existing student.

It then asks for the complete replacement information.

It creates:

```swift
let updatedData = NewStudent(...)
```

and sends:

```
PUT /students/{id}
```

PUT replaces the complete student data.

---

37. Menu Option 5 – Delete a Student

The application calls:

```swift
try await api.deleteStudent(id: id)
```

The API returns a message:

```swift
String
```

The message is displayed directly.

---

38. Menu Option 6 – Filter by Program

The current option performs local filtering.

First, it downloads every student:

```swift
let students =
    try await api.getStudents()
```

It then filters the array:

```swift
let result = students.filter {
    $0.program.lowercased() ==
    program.lowercased()
}
```

`filter` returns a new array containing only elements for which the condition is true.

The comparison is case-insensitive.

However, the program name must otherwise match exactly.

For example:

```
Computer Science
```

matches:

```
computer science
```

but does not match:

```
Computer
```

Local versus server-side filtering

The current menu downloads all students and filters them in Swift.

The project also contains:

```swift
getStudentsByProgram(program:)
```

This method asks Flask to filter the data.

For a small database, either approach works.

For a large database, server-side filtering is generally more efficient because fewer records travel over the network.

---

39. Menu Option 7 – Sort by Age

```swift
let sortedStudents = students.sorted {
    $0.age < $1.age
}
```

`sorted` creates a new array.

It does not modify the original array.

The closure compares two students.

`$0` represents the first student.

`$1` represents the second student.

The condition:

```swift
$0.age < $1.age
```

sorts from youngest to oldest.

To sort from oldest to youngest:

```swift
$0.age > $1.age
```

---

40. Menu Option 8 – Statistics

Option 8 retrieves every student:

```swift
let students =
    try await api.getStudents()
```

It passes the array to:

```swift
```

```
displayStatistics(students:)
```

The calculations happen locally in Swift.

The Flask API also has a dedicated endpoint for average age, but the current menu does not use it.

---

41. do, try, await, and catch

These concepts work together:

```swift
do {

    let students =
        try await api.getStudents()

} catch {

    print("Error: \(error)")
}
```

`do` begins a block whose errors will be handled.

`try` indicates that the called function can throw an error.

`await` waits for the asynchronous operation.

`catch` runs if an error is thrown.

PART H – VS CODE BUILD TASK

42. tasks.json

Create the folder:

```
.vscode
```

inside the project root.

Create:

```
.vscode/tasks.json
```

Add:

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "label": "Run Swift",
            "type": "shell",
            "command": "\"C:\\Program Files\\Microsoft Visual Studio\\18\\Community\\VC
\\Auxiliary\\Build\\vcvars64.bat\" && cd swift && swiftc main.swift Student.swift
APIService.swift APIError.swift -o app.exe && app.exe",
            "options": {
                "shell": {
                    "executable": "cmd.exe",
                    "args": [
                        "/d",
                        "/c"
                    ]
                }
            },
            "group": {
                "kind": "build",
                "isDefault": true
            },
            "problemMatcher": []
        }
    ]
}
```

---

43. Understanding tasks.json

43.1 Task Label

JSON

```
"label": "Run Swift"
```

This is the name displayed by VS Code.

43.2 Shell Task

JSON

```
"type": "shell"
```

VS Code executes a shell command.

43.3 Command

The command performs several operations:

```
vcvars64.bat
    &&
cd swift
    &&
swiftc main.swift Student.swift APIService.swift APIError.swift -o app.exe
    &&
app.exe
```

The `&&` operator means:

> Execute the next command only if the previous command succeeds.

43.4 vcvars64.bat

```
vcvars64.bat
```

loads the Visual Studio C++ compiler and Windows SDK environment required by Swift on Windows.

Without this environment, Swift may not find required Microsoft tools such as the linker.

43.5 cd swift

```
cd swift
```

moves from the project root into the folder containing the Swift files.

This is why VS Code must be opened on the project root.

43.6 swiftc

```
swiftc main.swift Student.swift APIService.swift APIError.swift -o app.exe
```

`swiftc` is the Swift compiler.

It compiles all four files together.

This is necessary because:

- `main.swift` uses `APIService`.
- `APIService.swift` uses `Student`, `NewStudent`, and `MessageResponse`.
- `APIService.swift` uses `APIError`.

Compiling only `main.swift` would cause errors such as:

```
Cannot find APIService in scope
```

The option:

```
-o app.exe
```

sets the executable filename.

43.7 Run the Executable

```
app.exe
```

starts the compiled console application.

43.8 Why Not Use `swift main.swift`?

The project is composed of several source files.

Running only:

```
Bash

swift main.swift
```

does not automatically include every other file.

The multi-file project must be compiled together:

```
Bash
```

```
swiftc main.swift Student.swift APIService.swift APIError.swift -o app.exe
```

On Windows, Swift also depends on the Visual Studio compiler environment.

The VS Code task performs both steps automatically.

---

44. Running the Complete Application

You need two terminals.

Terminal 1 – Flask

```bash
cd api
python app.py
```

Keep this terminal running.

Terminal 2 – Swift

Press:

```
Ctrl + Shift + B
```

Select:

```
Run Swift
```

The Swift console menu should appear.

---

45. Complete Execution Procedure

1. Open `LAB07_ADVANCED_REST_SWIFT` in VS Code.
2. Verify that `.vscode/tasks.json` exists.
3. Open the first terminal.
4. Run `cd api`.
5. Run `python app.py`.
6. Confirm that Flask is listening on port 5000.
7. Optionally test `GET /students` using Thunder Client.
8. Press `Ctrl + Shift + B`.
9. Run the Swift task.

10. Test menu option 1.
11. Add a student using option 3.
12. Display all students again.
13. Update a student using option 4.
14. Delete a student using option 5.
15. Filter students using option 6.
16. Sort students using option 7.
17. Display statistics using option 8.
18. Select option 0 to stop Swift.
19. Press `Ctrl + C` in the Flask terminal to stop the server.

---

46. What Is students.db?

`students.db` is a SQLite database file.

It is not a Python program.

It is not a Swift file.

It contains the actual stored rows from the `students` table.

The information remains after Flask stops.

This is called persistence.

If a student is added and you restart Flask, that student normally remains in the database.

The three starter students are inserted only when the table contains zero rows.

Deleting `students.db` resets the database, but this should only be done when a reset is intentionally required.

---

47. Common Errors and Solutions

Problem	Likely cause	Solution
`Network communication failed`	Flask is not running	Start `python app.py`
`Resource not found`	Student ID does not exist	Display all students and use a valid ID

Problem	Likely cause	Solution
`Invalid data sent to server`	Server returned status 400	Verify name, program, age and JSON types
`Cannot find Student in scope`	Only `main.swift` was compiled	Compile all four Swift files
`cd swift` fails	VS Code was opened inside the wrong folder	Open the project root
`vcvars64.bat` not found	Visual Studio path is different	Correct the path in `tasks.json`
Port 5000 is already used	Another Flask process is running	Stop the previous Flask process
Database appears empty	Flask is using another `students.db`	Start Flask from the `api` folder
Decoding error	JSON does not match Swift models	Inspect the response with Thunder Client
POST returns 400	JSON body is invalid	Use all three required properties
Swift works but Thunder Client does not	Thunder Client request is configured incorrectly	Verify URL, method, header and body
Thunder Client works but Swift does not	Problem is in Swift request or decoding	Examine `APIService.swift`

---

48. Learning Questions

1. Why does the Swift application not access SQLite directly?
2. What is the role of Flask?
3. What is the role of `APIService`?
4. Why does `NewStudent` not contain an ID?
5. What is the difference between `Student` and `NewStudent`?
6. Why does DELETE decode `MessageResponse`?
7. Why does POST return status 201?
8. Why can `URLSession` receive a valid response with status 404?

9. What does `Codable` provide?
10. What is the difference between `JSONEncoder` and `JSONDecoder`?
11. Why is `Content-Type: application/json` required?
12. What is the purpose of `async`?
13. What is the purpose of `await`?
14. What is the purpose of `throws`?
15. What does `try` mean?
16. Why are network calls placed inside `do-catch`?
17. What is the role of `validateResponse`?
18. Why must validation occur in Swift and Flask?
19. What does `map` do in `displayStatistics`?
20. What does `reduce` do?
21. Why are values converted to `Double` before division?
22. What is the difference between filtering locally and filtering on the server?
23. Why must all Swift files appear in the `swiftc` command?
24. What does `vcvars64.bat` provide?
25. Why must `.vscode` be created in the project root?
26. What is the purpose of `students.db`?
27. Why are SQL parameters safer than string concatenation?
28. What does `(id,)` mean in Python?
29. What happens if the Flask server is stopped while Swift is running?
30. Which part of the Swift project translates HTTP 404 into `APIError.notFound`?

---

49. Suggested Extensions

Extension 1 – Use Server-Side Program Filtering

Modify option 6 to call:

```swift
let students =
    try await api.getStudentsByProgram(
        program: program
    )
```

This avoids downloading every student.

Extension 2 – Add Average-Age Networking Method

Add the following method to `APIService`:

Swift

```
func getAverageAge()
async throws -> Double {

    guard let url = URL(
        string:
            "\(baseURL)/students/statistics/average-age"
    ) else {

        throw APIError.invalidURL
    }

    do {

        let (data, response) =
            try await URLSession.shared.data(
                from: url
            )

        try validateResponse(response)

        let result = try decode(
            AverageAgeResponse.self,
            from: data
        )

        return result.averageAge

    } catch let error as APIError {

        throw error

    } catch {

        throw APIError.networkError
    }
}
```

Menu option 8 could then request the average directly from Flask.

Extension 3 – Remove Redundant Code

After understanding the final code, identify and remove:

- The unused Python import.

- The unused first `decode` overload.
- The unused `APIService.validateStudent` method.

The application should still work after these elements are removed.

Extension 4 – Add Email

Add an email property to:

- The SQLite table.
- Flask validation.
- POST and PUT.
- `Student`.
- `NewStudent`.
- The console menu.

Extension 5 – Add PATCH

PUT replaces all student data.

Create a PATCH endpoint that allows a client to update only one property.

For example:

```JSON
{
  "age": 24
}
```

---

50. Submission Checklist

Before submitting, verify that:

- The project structure is correct.
- `.vscode` is inside the project root.
- `app.py` is inside `api`.
- `students.db` is inside `api`.
- The four Swift files are inside `swift`.
- Flask starts without errors.
- GET all works.

- GET by ID works.
- POST works.
- PUT works.
- DELETE works.
- Program filtering works.
- Average-age endpoint works.
- All Swift files compile together.
- Menu options 0 through 8 work.
- Invalid values produce readable messages.
- A missing ID produces a readable error.
- Thunder Client can test the API independently.
- The difference between the client, server, and database is understood.

Conclusion

This application is not merely a console program.

It is a complete client-server system.

The Swift console application is the client. It manages user interaction and sends HTTP requests.

Flask is the server. It receives requests, validates data, runs SQL commands, and returns JSON.

SQLite is the database. It stores the student information permanently.

Thunder Client is an independent testing client. It helps verify the Flask server before connecting it to Swift.

The most important execution path to understand is:

```
Menu choice
    ↓
main.swift
    ↓
APIService method
    ↓
URLSession request
    ↓
Flask route
    ↓
SQLite query
    ↓
```

```
JSON response
    ↓
validateResponse
    ↓
JSONDecoder
    ↓
Swift model
    ↓
Displayed result
```

A student who understands this complete path understands the central logic of REST API integration.